

CE QUE CETTE ANTISÈCHE RÉSOUT

Les trois zones où SQL renvoie **un résultat faux en silence** : jointures mal posées, NULL mal manipulé, alias utilisé trop tôt. À imprimer et garder sous la main.

Chaque module se lit seul. Les ★ signalent les pièges qui font le plus de dégâts en production.

01 · Jointures

02 · NULL

03 · Ordre d'exécution

★ = piège production

01 JOINTURES

intersection, extension, union

1.1 LES QUATRE TYPES

TYPE	CE QUE ÇA RENVOIE	USAGE TYPIQUE
INNER	Lignes présentes dans A et B (intersection)	Correspondances strictes, rien de plus
LEFT	Tout A + correspondances B (NULL si absente)	Garder tous les enregistrements source
RIGHT	Tout B + correspondances A (NULL si absente)	Symétrique du LEFT, rare en pratique
FULL OUTER	Tout A + tout B, NULL des deux côtés si absent	Réconciliation, diff de deux sources

1.2 ★ LE PIÈGE : LEFT JOIN TUÉ PAR LE WHERE

- ✗ Un LEFT JOIN **filtrer sur une colonne de B dans le WHERE** transforme silencieusement le LEFT en INNER : les lignes sans correspondance (B = NULL) sont éliminées.
- ✗ La condition d'exclusion doit aller **ON**, pas dans le **WHERE**.

```
-- ✗ Le LEFT JOIN est annulé (toutes les lignes B=NULL disparaissent)
SELECT a.*, b.montant
FROM commandes a
LEFT JOIN paiements b ON a.id = b.cmd_id
WHERE b.statut = 'validé'; -- ← tue le LEFT
```

```
-- ✓ La condition reste dans le ON : le LEFT est préservé
SELECT a.*, b.montant
FROM commandes a
LEFT JOIN paiements b
  ON a.id = b.cmd_id
 AND b.statut = 'validé'; -- ← dans le ON
```

1.3 RÉFLEXES JOINTURE

- ★ Toujours vérifier le **nombre de lignes après jointure** : un INNER peut vous coûter des lignes sans avertissement.
- Une clé de jointure non unique côté N produit des **doublons** : GROUP ou DISTINCT selon le besoin.
- Deux tables sans lien direct : passez par une **table de pont** plutôt qu'un produit cartésien.
- Le **CROSS JOIN** (produit cartésien) est rarement intentionnel : vérifiez la clause ON.

1.4 TESTER SES JOINTURES

- Étape 1** **Comptez avant/après** : SELECT COUNT(*) sur chaque table source, puis sur le résultat.
- Étape 2** **Cherchez les NULL inattendus** dans les colonnes de la table de droite après un LEFT JOIN.
- Étape 3** **Isolez une clé connue** : filtrez sur un ID précis et lisez chaque ligne du résultat.

2.1 NULL N'EST PAS UNE VALEUR

- ★ colonne = NULL ne marche **jamais** (renvoie toujours UNKNOWN, donc 0 ligne). Utiliser **IS NULL** ou **IS NOT NULL**.
NULL n'est pas 0, pas la chaîne vide '', pas false. C'est l'**absence de valeur**.
Toute comparaison avec NULL (=, !=, <, >) renvoie UNKNOWN : la ligne est exclue du résultat.

```
-- X Ne renvoie jamais rien (colonne = NULL est toujours UNKNOWN)
SELECT * FROM t WHERE email = NULL;

-- ✓ Syntaxe correcte
SELECT * FROM t WHERE email IS NULL;
SELECT * FROM t WHERE email IS NOT NULL;
```

2.2 ★ NOT IN AVEC NULL = 0 LIGNE

- ✗ Si la sous-requête **NOT IN** pour toute la requête. SQL ne peut pas prouver que la valeur n'est pas dans un ensemble qui contient un inconnu.
- ✗ Remplacer **NOT** ou filtrer les WHERE id dans la par **EXISTS** NULL avec IS NOT sous-requête.

```
-- X Renvoie 0 ligne si sous-requête contient un NULL
SELECT * FROM clients
WHERE id NOT IN (SELECT client_id FROM commandes);

-- ✓ NOT EXISTS : robuste aux NULL
SELECT * FROM clients c
WHERE NOT EXISTS (
    SELECT 1 FROM commandes o
    WHERE o.client_id = c.id);
```

2.3 NULL DANS LES AGRÉGATS

FONCTION	COMPORTEMENT AVEC NULL
COUNT(*)	Compte toutes les lignes, NULL inclus
COUNT(col)	Ignore les NULL : compte seulement les valeurs renseignées
SUM(col)	Ignore les NULL (ne les traite pas comme 0)
AVG(col)	Ignore les NULL au numérateur ET au dénominateur
MIN / MAX	Ignorent les NULL
RÉFLEXE	
« Si COUNT(*) ≠ COUNT(col), la colonne contient des NULL : vérifiez si c'est voulu. »	

2.4 GÉRER LES NULL À L'AFFICHAGE

```
-- Remplacer NULL par une valeur de repli
COALESCE(colonne, 'valeur par défaut')
COALESCE(montant, 0)

-- Tester NULL dans une expression
CASE WHEN colonne IS NULL THEN 'absent'
ELSE colonne END
```

2.5 LE KEY-MOMENT NULL

LA RÈGLE NULL EN 3 LIGNES

1. **Comparer** : toujours IS NULL / IS NOT NULL, jamais = NULL.
2. **NOT IN** : si la sous-requête peut contenir des NULL, préférer NOT EXISTS.
3. **Agréger** : COUNT(*) compte tout, COUNT(col) saute les NULL, SUM les ignore (pas de 0).

03 ORDRE D'EXÉCUTION LOGIQUE

ce que SQL lit n'est pas ce qu'il exécute

3.1 L'ORDRE RÉEL D'ÉVALUATION

1. **FROM** Identifier les tables sources (et les produits cartésiens éventuels)
2. **JOIN** Appliquer les conditions de jointure (ON), constituer l'ensemble de lignes
3. **WHERE** Filtrer ligne par ligne (avant toute agrégation)
4. **GROUP BY** Regrouper les lignes filtrées
5. **HAVING** Filtrer les groupes (après agrégation)
6. **SELECT** Calculer les colonnes et les alias du résultat
7. **ORDER BY** Trier (ici les alias SELECT sont disponibles)
8. **LIMIT** Tronquer le résultat final

3.2 ★ CONSÉQUENCE 1 : LES ALIAS NE SONT PAS DANS LE WHERE

- ✗ Le **après** le WHERE : un alias défini dans le SELECT n'existe pas encore au moment où le est WHERE s'exécute.

```
-- X Erreur : 'ca_ttc' n'existe pas au niveau WHERE
SELECT montant * 1.2 AS ca_ttc
FROM ventes
WHERE ca_ttc > 1000; -- ← colonne inconnue

-- ✓ Répéter l'expression, ou encapsuler dans une sous-requête
SELECT montant * 1.2 AS ca_ttc
FROM ventes
WHERE montant * 1.2 > 1000;

-- ✓ Ou via CTE / sous-requête
WITH base AS (
    SELECT montant * 1.2 AS ca_ttc FROM ventes)
SELECT * FROM base WHERE ca_ttc > 1000;
```

3.3 ★ CONSÉQUENCE 2 : WHERE AVANT L'AGRÉGAT, HAVING APRÈS

- ✗ **WHERE** filtre les lignes individuelles **avant** le GROUP BY : il ne peut pas porter sur un agrégat (SUM , COUNT ...).
- ✗ **HAVING** filtre les groupes **après** le GROUP BY : il s'applique aux agrégats.

```
-- ✗ WHERE ne peut pas filtrer sur un agrégat
SELECT client_id, SUM(montant) AS total
FROM commandes
GROUP BY client_id
WHERE total > 500; -- ← erreur syntaxe
```

```
-- ✓ HAVING pour les conditions post-agrégat
SELECT client_id, SUM(montant) AS total
FROM commandes
GROUP BY client_id
HAVING SUM(montant) > 500;
```

3.4 WHERE VS HAVING : AIDE-MÉMOIRE

CLAUSE	QUAND ?	SUR QUOI ?	EXEMPLE
WHERE	Avant GROUP BY	Colonnes brutes, pas d'agrégats	WHERE statut = 'actif'
HAVING	Après GROUP BY	Résultats d'agrégats	HAVING COUNT(*) > 3

3.5 RÉCAPITULATIF VISUEL DE L'ORDRE

L'ORDRE À GARDER EN TÊTE

1. **FROM · JOIN** : on constitue le jeu de lignes.
2. **WHERE** : on filtre les lignes (pas encore d'agrégat).
3. **GROUP BY · HAVING** : on regroupe, puis on filtre les groupes.
4. **SELECT** : on calcule les colonnes et les alias.
5. **ORDER BY · LIMIT** : on trie et on tronque.

→ Alias du SELECT : disponibles dans ORDER BY, PAS dans WHERE ni HAVING.